FinancePro

Sistema de Gestão Financeira Pessoal com Inteligência Artificial

Documentação Técnica Completa

Versão 1.0 — MVP

Stack principal:	Django 4.2 · Python 3.x · SQLite
IA / NLP:	LangChain 0.3 · OpenAI GPT-4o-mini
Frontend:	HTML5 · CSS3 · JavaScript · Chart.js
Idioma:	Português (Brasil) · pt-BR
Licença:	Proprietária

Documentação gerada em 2 de abril de 2026

Resumo Executivo

O **FinancePro** é um sistema web de gestão financeira pessoal desenvolvido em **Django 4.2**, voltado exclusivamente ao mercado brasileiro. A plataforma permite ao usuário cadastrar receitas, gastos fixos e variáveis, reservas de emergência e investimentos, consolidando todos os dados em um painel interativo com gráficos em tempo real.

O diferencial central do sistema é a integração com **cinco módulos de Inteligência Artificial** construídos sobre a arquitetura LangChain + OpenAI GPT-4o-mini, que transformam dados financeiros brutos em diagnósticos precisos, recomendações acionáveis e simulações de cenários hipotéticos com valores em Reais.

Principais funcionalidades:

- Gestão completa de finanças pessoais (CRUD) com autenticação via e-mail
- Dashboard interativo com KPIs calculados automaticamente e checklist de pagamentos
- **AI Financial Coach** — pontuação de saúde financeira e recomendações personalizadas
- **AI Spending Analysis** — detecção de desperdícios, padrões ruins e riscos financeiros
- **AI Budget Optimizer** — otimização orçamentária baseada na Regra 50/30/20
- **AI Investment Assistant** (Premium) — análise de portfólio, perfil de risco e projeções de patrimônio
- **AI What-if Simulator** — chatbot conversacional para simulação de cenários hipotéticos

Sumário

Resumo Executivo **2**

1 Visão Geral do Projeto **3**
1.1 Contexto e Motivação 3
1.2 Objetivos do Sistema 3
1.3 Stack Tecnológica 4
1.4 Estrutura de Diretórios 4

2 Arquitetura do Sistema **7**
2.1 Diagrama de Arquitetura Geral 7

2.2 Padrão Arquitetural . . . 7
2.3 Padrão dos Agentes de IA . . . 8
2.4 Fluxo de Dados . . . 9

3 Configuração e Instalação . . . 10
3.1 Pré-requisitos . . . 10
3.2 Instalação Rápida . . . 10
3.3 Instalação Manual . . . 10
3.4 Variáveis de Ambiente . . . 11
3.5 Dependências do Projeto . . . 12

4 Módulo de Autenticação . . . 13
4.1 Modelo de Usuário Customizado . . . 13
4.2 Formulários de Autenticação . . . 14
4.3 URLs de Autenticação . . . 14

5 Módulo Financeiro — Modelos de Dados . . . 15
5.1 Diagrama Entidade-Relacionamento . . . 15
5.2 Detalhamento dos Modelos . . . 15
5.2.1 Receita . . . 15
5.2.2 GastoFixo . . . 16
5.2.3 GastoVariavel . . . 16
5.2.4 Reserva . . . 17
5.2.5 Investimento . . . 17

6 Módulo Financeiro — Views e URLs . . . 18
6.1 Dashboard Principal . . . 18
6.2 Tabela de Rotas Completa . . . 19
6.3 Endpoint AJAX: Toggle de Pagamento . . . 20

7 Módulos de Inteligência Artificial . . . 21
7.1 Visão Geral dos Agentes . . . 21
7.2 Construtor de Contexto Financeiro . . . 21
7.3 AI Financial Coach . . . 22
7.3.1 Objetivo . . . 22
7.3.2 Modelos de Saída (Pydantic) . . . 22
7.3.3 Lógica de Pontuação . . . 23
7.3.4 Integração com o Dashboard . . . 23
7.4 AI Spending Analysis . . . 23
7.4.1 Objetivo . . . 23
7.4.2 Modelos de Saída (Pydantic) . . . 23
7.4.3 Padrões Detectados . . . 24
7.5 AI Budget Optimizer . . . 24
7.5.1 Objetivo . . . 24
7.5.2 Regra 50/30/20 Aplicada . . . 25
7.5.3 Modelos de Saída (Pydantic) . . . 25
7.5.4 Cálculo de Projeções . . . 26
7.6 AI Investment Assistant . . . 26

7.6.1 Objetivo . . . 26
7.6.2 Perfis de Risco e Alocações Ideais . . . 26
7.6.3 Modelos de Saída (Pydantic) . . . 26
7.6.4 Fórmula de Projeção . . . 27
7.7 AI What-if Simulator . . . 27
7.7.1 Objetivo . . . 27
7.7.2 Gerenciamento do Histórico de Conversa . . . 28
7.7.3 Modelos de Saída (Pydantic) . . . 28
7.7.4 Exemplos de Cenários Suportados . . . 29
7.7.5 Sistema de Veredictos . . . 29

8 Interface do Usuário . . . 30
8.1 Sistema de Design . . . 30
8.1.1 Paleta de Cores . . . 30
8.1.2 Componentes Reutilizáveis . . . 30
8.2 Layout Principal . . . 31
8.3 Template Base . . . 31
8.4 Páginas de Autenticação . . . 32
8.5 Interfaces dos Módulos de IA . . . 32

9 Segurança . . . 33
9.1 Mecanismos Implementados . . . 33
9.2 Pontos de Atenção para Produção . . . 34
9.3 Checklist de Segurança para Deploy . . . 34

10 Deploy e Produção . . . 35
10.1 Requisitos de Servidor . . . 35
10.2 Stack de Produção Recomendada . . . 35
10.3 Variáveis de Ambiente de Produção . . . 36

11 Migrações do Banco de Dados . . . 37
11.1 Histórico de Migrações . . . 37
11.2 Comandos de Migração . . . 37

12 Guia de Extensão . . . 38
12.1 Adicionando um Novo Modelo Financeiro . . . 38
12.2 Adicionando um Novo Agente de IA . . . 38
12.3 Alterando o Modelo de LLM . . . 38

13 Referências e Tecnologias . . . 40

Capítulo 1

Visão Geral do Projeto

1.1 Contexto e Motivação

O planejamento financeiro pessoal é um desafio recorrente para a maioria dos brasileiros. A ausência de ferramentas acessíveis, integradas e inteligentes que falem a língua do cidadão comum — em Reais, com contexto cultural brasileiro — motivou o desenvolvimento do FinancePro.

O sistema foi concebido como um **MVP (Minimum Viable Product)** funcional e extensível, com uma base técnica sólida que permite evolução para produto SaaS completo sem refatoração estrutural significativa.

1.2 Objetivos do Sistema

1. Oferecer controle financeiro completo em uma única plataforma
2. Aplicar Inteligência Artificial para transformar dados em insights acionáveis
3. Calcular impactos financeiros concretos (R$, %, prazos em meses/anos)
4. Ser 100% localizado para o contexto brasileiro (moeda, benchmarks, linguagem)
5. Manter arquitetura limpa e expansível para futuras funcionalidades

1.3 Stack Tecnológica

Tabela 1.1: Tecnologias utilizadas no FinancePro

Camada	Tecnologia	Descrição
Backend	Django 4.2.29	Framework web Python com ORM, autenticação e admin
Linguagem	Python 3.x	Linguagem de programação principal
Banco de dados	SQLite3	Banco relacional embutido (dev); compatível com PostgreSQL
IA / NLP	LangChain $\geq$ 0.3	Orquestração de agentes e cadeias de processamento
LLM	OpenAI GPT-4o-mini	Modelo de linguagem para análises financeiras
Validação	Pydantic v2	Modelos de saída estruturada dos agentes IA
Variáveis de ambiente	python-dotenv 1.0	Carregamento seguro de configurações
Frontend	HTML5 + CSS3 + JS	Interface responsiva sem frameworks JS pesados
Gráficos	Chart.js 4.4	Gráficos interativos (donut, barra)
Ícones	Font Awesome 6.5	Ícones vetoriais em toda a interface
Tipografia	Google Fonts (Inter)	Fonte principal da interface

1.4 Estrutura de Diretórios

Listing 1.1: Estrutura completa do projeto

```
MVP - Finance Django/
        .env                          # Vari veis de
  ambiente (API keys, SECRET_KEY)
        .venv/                        # Ambiente virtual
  Python
        db.sqlite3                    # Banco de dados SQLite
   (desenvolvimento)
        manage.py                     # CLI do Django
        requirements.txt              # Depend ncias
  m nimas de produ   o
```

```
        requirements_django.txt      # Depend ncias
completas (com IA)
        setup.sh                     # Script de
configura  o inicial

        finance_project/             # Configura  es do
projeto Django
            settings.py                # Configura  es
globais
            urls.py                    # Roteamento raiz
            wsgi.py                    # Interface WSGI para
 produ  o

        accounts/                    # App de autentica  o
            migrations/
            admin.py
            forms.py                   # Formul rios de
login, registro, perfil
            models.py                  # Modelo de usu rio
customizado
            urls.py
            views.py

        finance/                     # App principal de
finan as
            migrations/
            admin.py
            forms.py                   # Formul rios
financeiros
            models.py                  # Modelos: Receita,
Gastos, Reservas, Investimentos
            urls.py
            views.py                   # Views CRUD +
endpoints de IA
            ai_coach.py                # Agente IA: Coach
Financeiro
            budget_optimizer.py      # Agente IA:
Otimizador de Or amento
            investment_assistant.py # Agente IA:
Assistente de Investimentos
            spending_analyzer.py     # Agente IA:
Analisador de Gastos
            whatif_simulator.py      # Agente IA:
Simulador What-if (chatbot)

        templates/
            base.html                # Template base (
    sidebar, header)
            accounts/                # Templates de
    autentica  o
```

```
        finance/                    # Templates de
    finanças e IA
```

Capítulo 2

Arquitetura do Sistema

2.1 Diagrama de Arquitetura Geral

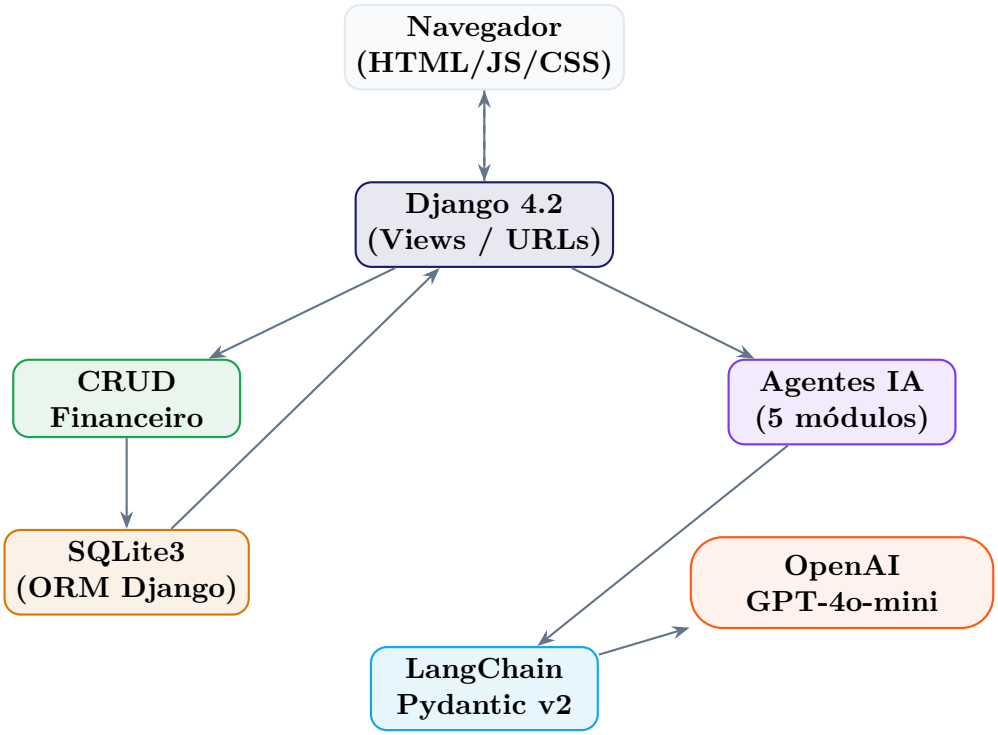


Figura 2.1: Diagrama de arquitetura do FinancePro

2.2 Padrão Arquitetural

O sistema adota o padrão **MTV (Model-Template-View)** do Django, com extensão para agentes de IA:

Model Modelos de dados mapeados via ORM do Django para o banco SQLite. Cada entidade financeira possui seu próprio modelo com validações, relacionamentos e propriedades calculadas.

Template Templates HTML com herança hierárquica partindo de `base.html`. Toda a interface usa CSS variables, Font Awesome 6.5 e Chart.js, sem dependência de frameworks JavaScript externos.

View Views Python com decoradores `@login_required`. Endpoints CRUD retornam templates renderizados; endpoints de IA retornam `JsonResponse` para consumo assíncrono via `fetch()`.

AI Layer Cinco módulos independentes, cada um seguindo um padrão uniforme: modelos Pydantic → `PromptTemplate` → `ChatOpenAI` → `JsonOutputParser`.

2.3 Padrão dos Agentes de IA

Todos os cinco módulos de IA seguem exatamente a mesma arquitetura de classe, garantindo consistência e facilidade de manutenção:

```python
from pydantic import BaseModel, Field
from langchain_core.output_parsers import JsonOutputParser
from langchain_core.prompts import PromptTemplate
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv

class OutputModel(BaseModel):
    campo: str = Field(description="Descri  o do campo")

class AgentClass:
    def __init__(self, model=None):
        load_dotenv()
        self.model = model or ChatOpenAI(model="gpt-4o-mini",
            temperature=X)
        self.parser = JsonOutputParser(pydantic_object=
           OutputModel)
        self.prompt = self._build_prompt()
        self.chain  = self._build_chain()

    def _build_prompt(self) -> PromptTemplate:
        return PromptTemplate(
            template=TEMPLATE,
            input_variables=["financial_data", ...],
            partial_variables={
                "format_instructions": self.parser.
                   get_format_instructions()
            },
        )

    def _build_chain(self):
        return self.prompt | self.model | self.parser

    def analyse(self, user) -> dict:
        context = _build_context(user)
        return self.chain.invoke({"financial_data": context})
```

Listing 2.1: Padrão base de todos os agentes IA

2.4 Fluxo de Dados

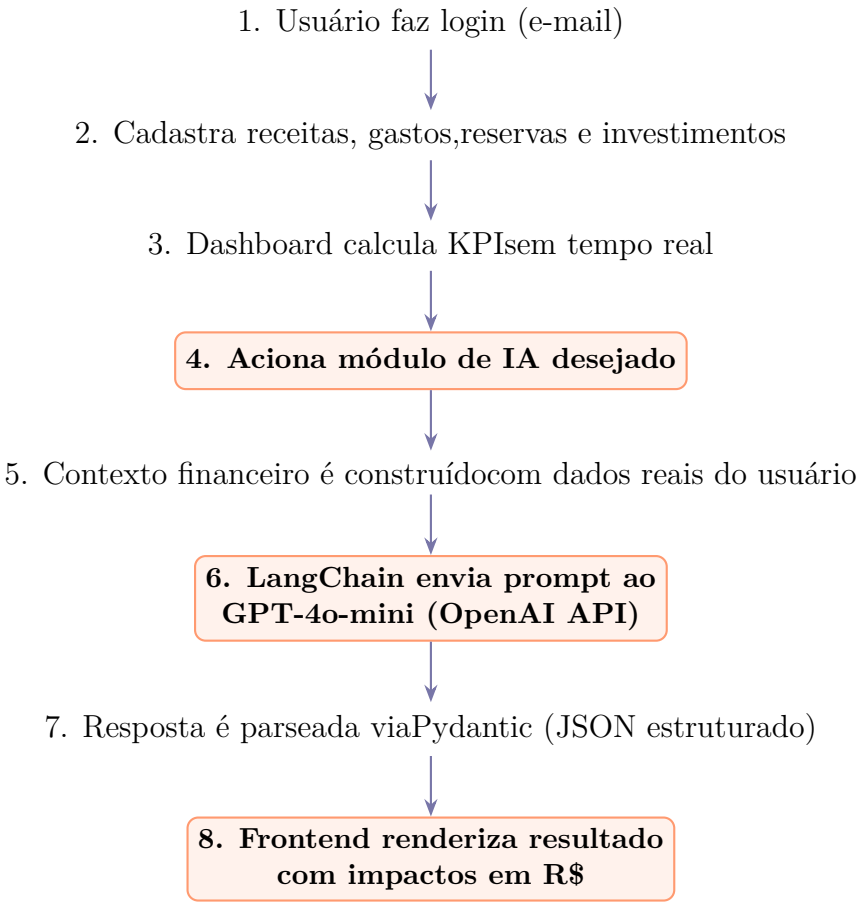


Figura 2.2: Fluxo completo de dados no FinancePro

Capítulo 3

Configuração e Instalação

3.1 Pré-requisitos

- Python 3.10 ou superior
- pip3 / virtualenv
- Chave de API da OpenAI (`OPENAI_API_KEY`)
- Acesso à internet (para chamadas à API OpenAI)

3.2 Instalação Rápida

Listing 3.1: Instalação via script automatizado

```
# Clone ou extraia o projeto
cd "MVP - Finance Django"

# Execute o script de configura  o
bash setup.sh
```

O script `setup.sh` executa automaticamente:

1. Instalação de todas as dependências via `pip3 install -r requirements_django.txt`
2. Aplicação de todas as migrações do banco de dados
3. Criação do diretório `static/`
4. Criação interativa do superusuário (administrador)

3.3 Instalação Manual

Listing 3.2: Instalação passo a passo

```
# 1. Criar e ativar ambiente virtual
python3 -m venv .venv
source .venv/bin/activate          # Linux/macOS
.venv\Scripts\activate             # Windows

# 2. Instalar depend ncias
pip3 install -r requirements_django.txt
```

```
# 3. Configurar variaveis de ambiente
cp .env.example .env                # Editar com seus valores

# 4. Aplicar migracoes
python manage.py migrate

# 5. Criar superusuario
python manage.py createsuperuser

# 6. Iniciar servidor de desenvolvimento
python manage.py runserver
```

3.4 Variáveis de Ambiente

O arquivo `.env` na raiz do projeto controla todas as configurações sensíveis:

Tabela 3.1: Variáveis de ambiente do sistema

Variável	Obrigatória	Descrição
`SECRET_KEY`	Sim	Chave secreta do Django para criptografia de sessão
`DEBUG`	Não	`True` (dev) ou `False` (produção)
`ALLOWED_HOSTS`	Não	Hosts permitidos, separados por vírgula
`OPENAI_API_KEY`	Sim	Chave da API OpenAI para os módulos de IA

Aviso de Segurança

Nunca comite o arquivo `.env` em repositórios públicos. Adicione-o ao `.gitignore`. A chave `OPENAI_API_KEY` representa créditos financeiros reais e, se exposta, pode ser explorada por terceiros.

3.5 Dependências do Projeto

Tabela 3.2: Dependências principais

Pacote	Versão	Finalidade
`Django`	$\geq 4.2, < 5.0$	Framework web principal
`Pillow`	≥ 10.0	Processamento de imagens (foto de perfil)
`python-dotenv`	≥ 1.0	Carregamento do arquivo `.env`
`langchain`	≥ 0.3	Orquestração de agentes LLM
`langchain-openai`	≥ 0.2	Integração LangChain com OpenAI
`openai`	≥ 1.0	SDK oficial da OpenAI
`pydantic`	v2 (via langchain)	Validação e serialização de modelos

Capítulo 4

Módulo de Autenticação

4.1 Modelo de Usuário Customizado

O sistema utiliza um **modelo de usuário completamente customizado**, substituindo o padrão do Django (que usa *username* como campo primário) por autenticação baseada em **e-mail**.

```
class User(AbstractBaseUser, PermissionsMixin):
    email       = models.EmailField(unique=True)   # Campo de login
    first_name  = models.CharField(max_length=100)
    last_name   = models.CharField(max_length=100, blank=True)
    photo       = models.ImageField(upload_to='avatars/', blank=
       True, null=True)
    is_active   = models.BooleanField(default=True)
    is_staff    = models.BooleanField(default=False)
    date_joined = models.DateTimeField(auto_now_add=True)

    USERNAME_FIELD  = 'email'       # Login por e-mail
    REQUIRED_FIELDS = ['first_name']

    objects = UserManager()
```

Listing 4.1: accounts/models.py — Modelo User customizado

```
AUTH_USER_MODEL = 'accounts.User'
LOGIN_URL = '/accounts/login/'
LOGIN_REDIRECT_URL = '/dashboard/'
LOGOUT_REDIRECT_URL = '/accounts/login/'
```

Listing 4.2: settings.py — Apontando para o modelo customizado

4.2 Formulários de Autenticação

Tabela 4.1: Formulários do módulo accounts

Formulário	Campos e Validações
`LoginForm`	`email` (EmailField) + `password` (PasswordInput)
`RegisterForm`	email, first_name, last_name, password, password2 (confirmação)
`ProfileForm`	first_name, last_name, email, photo
`ChangePasswordForm`	old_password, new_password (mín. 8 chars), confirm_password

4.3 URLs de Autenticação

Tabela 4.2: Rotas do app accounts

URL	Método	Função
`/accounts/login/`	GET + POST	Exibe e processa o formulário de login
`/accounts/register/`	GET + POST	Cadastro de novos usuários
`/accounts/logout/`	POST	Encerra a sessão do usuário
`/accounts/profile/`	GET + POST	Edição de perfil e troca de senha

Capítulo 5

Módulo Financeiro — Modelos de Dados

5.1 Diagrama Entidade-Relacionamento

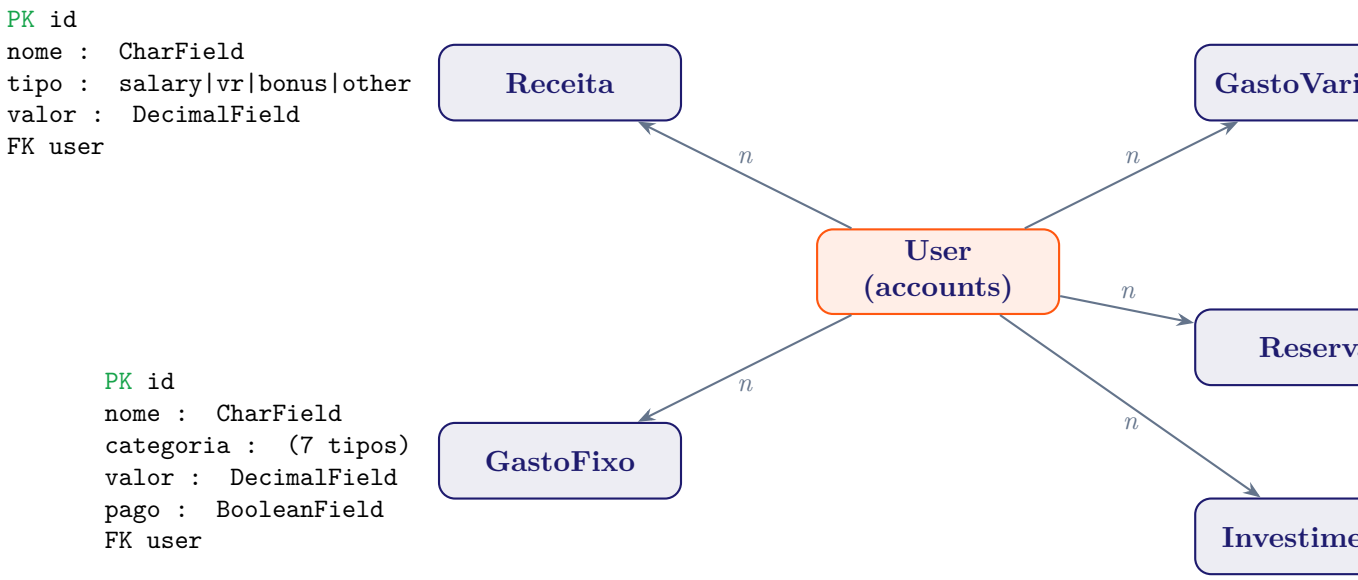


Figura 5.1: Diagrama Entidade-Relacionamento do FinancePro

5.2 Detalhamento dos Modelos

5.2.1 Receita

Representa qualquer fonte de renda do usuário.

Tabela 5.1: Campos do modelo Receita

Campo	Tipo	Descrição
`nome`	CharField(200)	Nome descritivo da receita
`tipo`	CharField(choices)	salary, vr, bonus, other
`valor`	DecimalField(10,2)	Valor mensal em Reais
`user`	ForeignKey(User)	Proprietário do registro
`criado`	DateTimeField(auto)	Timestamp de criação

5.2.2 GastoFixo

Despesas mensais recorrentes com valor fixo. Possui o campo `pago` para controle de pagamento mensal (checklist no dashboard).

Tabela 5.2: Campos do modelo GastoFixo

Campo	Tipo	Descrição
`nome`	CharField(200)	Nome do gasto fixo
`categoria`	CharField(choices)	moradia, transporte, saude, educacao, servicos, seguros, outros
`valor`	DecimalField(10,2)	Valor mensal em Reais
`pago`	BooleanField	Status de pagamento do mês corrente
`user`	ForeignKey(User)	Proprietário do registro
`criado`	DateTimeField(auto)	Timestamp de criação

5.2.3 GastoVariavel

Despesas variáveis sem valor fixo mensal. Permite rastrear gastos por data.

Tabela 5.3: Campos do modelo GastoVariavel

Campo	Tipo	Descrição
`nome`	CharField(200)	Nome do gasto variável
`categoria`	CharField(choices)	alimentacao, lazer, vestuario, eletronicos, higiene, presentes, outros
`valor`	DecimalField(10,2)	Valor gasto
`data`	DateField	Data do gasto
`user`	ForeignKey(User)	Proprietário do registro
`criado`	DateTimeField(auto)	Timestamp de criação

5.2.4 Reserva

Representa fundos de emergência ou poupanças com meta definida. Possui propriedades calculadas para exibição de progresso.

Tabela 5.4: Campos do modelo Reserva

Campo	Tipo	Descrição
`nome`	CharField(200)	Nome da reserva
`valor`	DecimalField(10,2)	Aporte mensal
`meta`	DecimalField(10,2)	Valor-alvo da reserva (opcional)
`acumulado`	DecimalField(10,2)	Total já acumulado
`user`	ForeignKey(User)	Proprietário do registro
`progresso`	*property*	% de conclusão em relação à meta
`progresso_cor`	*property*	Cor CSS baseada no progresso

5.2.5 Investimento

Registra posições de investimento com aporte mensal e total acumulado.

Tabela 5.5: Campos do modelo Investimento

Campo	Tipo	Descrição
`nome`	CharField(200)	Nome do investimento
`tipo`	CharField(choices)	renda_fixa, renda_variavel, fii, cripto, previdencia, outros
`valor`	DecimalField(10,2)	Aporte mensal em Reais
`total`	DecimalField(10,2)	Total já investido
`user`	ForeignKey(User)	Proprietário do registro

Capítulo 6

Módulo Financeiro — Views e URLs

6.1 Dashboard Principal

O dashboard é a view central do sistema, calculando automaticamente todos os KPIs financeiros do usuário.

```
def _totals(user):
    rec  = user.receitas.aggregate(t=Sum('valor'))['t'] or Decimal('0')
    fix  = user.gastos_fixos.aggregate(t=Sum('valor'))['t'] or Decimal('0')
    vari = user.gastos_variaveis.aggregate(t=Sum('valor'))['t'] or Decimal('0')
    res  = user.reservas.aggregate(t=Sum('valor'))['t'] or Decimal('0')
    inv  = user.investimentos.aggregate(t=Sum('valor'))['t'] or Decimal('0')
    gas  = fix + vari + res + inv
    sal  = rec - gas
    return {'rec': rec, 'fix': fix, 'vari': vari,
            'res': res, 'inv': inv, 'gas': gas, 'sal': sal}
```

Listing 6.1: finance/views.py — Helper de totais

KPIs calculados no dashboard:

- Total de receitas mensais
- Total de gastos (fixos + variáveis + reservas + investimentos)
- Saldo disponível (receitas − todos os gastos)
- Aportes mensais em reservas e investimentos
- Checklist de pagamento dos gastos fixos (com progresso em tempo real)

Gráficos gerados:

- **Donut (Chart.js):** Distribuição dos gastos por categoria com percentuais
- **Barras (Chart.js):** Alocação percentual em relação à renda total

6.2 Tabela de Rotas Completa

Tabela 6.1: Todas as rotas do app finance

URL	Método	Descrição
/dashboard/	GET	Dashboard principal com KPIs
Receitas		
/receitas/	GET POST	Listar + adicionar receita
/receitas/{pk}/editar/	GET POST	Editar receita
/receitas/{pk}/excluir/	GET POST	Excluir receita
Gastos Fixos		
/gastos-fixos/	GET POST	Listar + adicionar gasto fixo
/gastos-fixos/{pk}/editar/	GET POST	Editar gasto fixo
/gastos-fixos/{pk}/excluir/	GET POST	Excluir gasto fixo
/gastos-fixos/{pk}/toggle-pago/	POST	Toggle status pago (AJAX)
Gastos Variáveis		
/gastos-variaveis/	GET POST	Listar + adicionar
/gastos-variaveis/{pk}/editar/	GET POST	Editar
/gastos-variaveis/{pk}/excluir/	GET POST	Excluir
Reservas		
/reservas/	GET POST	Listar + adicionar
/reservas/{pk}/editar/	GET POST	Editar
/reservas/{pk}/excluir/	GET POST	Excluir
Investimentos		
/investimentos/	GET POST	Listar + adicionar
/investimentos/{pk}/editar/	GET POST	Editar
/investimentos/{pk}/excluir/	GET POST	Excluir
Módulos de IA		
/ai-coach/	POST	Análise do Coach (JSON)
/spending-analysis/	GET	Página de análise de gastos
/spending-analysis/run/	POST	Executa análise (JSON)
/budget-optimizer/	GET	Página do otimizador
/budget-optimizer/run/	POST	Executa otimização (JSON)
/investment-assistant/	GET	Página do assistente
/investment-assistant/run/	POST	Executa análise (JSON)
/whatif-simulator/	GET	Página do chatbot
/whatif-simulator/chat/	POST	Envia mensagem ao chatbot

6.3 Endpoint AJAX: Toggle de Pagamento

Um dos recursos mais interativos do sistema é o checklist de gastos fixos no dashboard. Ao clicar em um item, uma requisição AJAX é enviada sem recarregar a página:

```
@login_required
def toggle_gasto_fixo_pago(request, pk):
    if request.method != 'POST':
        return JsonResponse({'error': 'method not allowed'}, status=405)
    obj = get_object_or_404(GastoFixo, pk=pk, user=request.user)
    obj.pago = not obj.pago
    obj.save()
    all_fixos = request.user.gastos_fixos.all()
    return JsonResponse({
        'pago': obj.pago,
        'pago_total': float(all_fixos.filter(pago=True).aggregate(
                            t=Sum('valor'))['t'] or 0),
        'pendente_total': float(all_fixos.filter(pago=False).aggregate(
                                t=Sum('valor'))['t'] or 0),
        'pago_count': all_fixos.filter(pago=True).count(),
        'total_count': all_fixos.count(),
    })
```

Listing 6.2: View de toggle de pagamento

Capítulo 7

Módulos de Inteligência Artificial

7.1 Visão Geral dos Agentes

O FinancePro possui cinco agentes de IA independentes, todos integrados ao mesmo ecossistema LangChain + OpenAI. Cada agente recebe o contexto financeiro completo do usuário e retorna uma resposta estruturada via `JsonOutputParser` com modelos Pydantic.

Tabela 7.1: Comparativo dos cinco agentes de IA

Módulo	Classe	Temp.	Foco principal
AI Coach	`FinancialCoach`	0.6	Pontuação de saúde e insights gerais
Spending Analysis	`SpendingAnalyzer`	0.5	Detecção de desperdícios e riscos
Budget Optimizer	`BudgetOptimizer`	0.4	Alocação ideal 50/30/20
Investment AI	`InvestmentAssistant`	0.3	Portfólio, risco e projeções
What-if Simulator	`WhatIfSimulator`	0.7	Chatbot de cenários hipotéticos

Dica

A temperature é reduzida progressivamente dos módulos mais *conversacionais* (Coach, What-if) para os mais *analíticos* (Investimentos), garantindo maior precisão nas projeções financeiras.

7.2 Construtor de Contexto Financeiro

Todos os agentes utilizam uma função específica para montar o contexto do usuário em texto estruturado. O padrão geral é:

```
=== SITUA  O FINANCEIRA ATUAL ===
Renda mensal total:          R$ 8.500,00
Total gastos mensais:        R$ 6.200,00 (72.9% da renda)
  -> Gastos fixos:           R$ 3.200,00 (37.6%)
  -> Gastos vari veis:        R$ 1.500,00 (17.6%)
  -> Aportes reservas:       R$ 800,00   (9.4%)
```

```
  -> Aportes investimentos:R$ 700,00   (8.2%)
SALDO LIVRE MENSAL:        R$ 2.300,00 (27.1% da renda)
Patrim nio total:           R$ 42.000,00
  -> Reservas acumuladas:  R$ 18.000,00
  -> Investimentos:        R$ 24.000,00

=== RECEITAS ===
      Sal rio CLT (Sal rio): R$ 8.000,00/m s
      Vale Refei  o (VR/VA): R$ 500,00/m s

=== GASTOS FIXOS MENSAIS ===
      Aluguel (Moradia): R$ 1.800,00
  ...

=== M TRICAS PARA C LCULO ===
  Reserva emerg ncia ideal (6x desp.): R$ 28.200,00
  Cobertura atual:                     63.8%
  Taxa de poupan a:                     17.6%
  Comprometimento da renda:            72.9%
  Margem dispon vel para novos gastos: R$ 2.300,00/m s
```

Listing 7.1: Estrutura do contexto financeiro enviado ao LLM

7.3 AI Financial Coach

7.3.1 Objetivo

Analisar a saúde financeira geral do usuário, atribuir uma **pontuação de 0 a 100** e gerar insights personalizados com recomendações acionáveis.

7.3.2 Modelos de Saída (Pydantic)

```python
class Insight(BaseModel):
    tipo:     str  # 'positivo', 'aviso', 'critico'
    icone:    str  # Emoji representativo
    titulo:   str  # T tulo curto do insight
    mensagem: str  # Explica  o com dados espec ficos em
       R$

class FinancialAnalysis(BaseModel):
    score:         int          # 0-100
    score_label:   str          # 'Excelente', 'Bom', 'Regular
       ', 'Cr tico'
    score_color:   str          # Cor hex (#16A34A, #F59E0B,
       etc.)
    resumo:        str          # Diagn stico geral em 2-3
       frases
```

```
    insights:       List[Insight]   # 3-5 insights
    recomendacoes: List[str]    # Exatamente 3 recomenda  es
        acion veis
```

Listing 7.2: finance/ai_coach.py — Modelos Pydantic

7.3.3 Lógica de Pontuação

A IA é instruída a calcular a pontuação com base nos seguintes benchmarks brasileiros:

Tabela 7.2: Benchmarks de pontuação de saúde financeira

Critério	Referência	Impacto
Gastos fixos / renda	$\leq 50\%$	Alto
Gastos variáveis / renda	$\leq 20\%$	Médio
Taxa de poupança	$\geq 20\%$	Alto
Reserva de emergência	$\geq 6\times$ gastos mensais	Alto
Saldo livre mensal	> 0	Crítico
Diversificação de investimentos	> 1 tipo	Médio

7.3.4 Integração com o Dashboard

O AI Coach é o único agente integrado diretamente no dashboard, sem página própria. O botão *"Analisar minhas finanças"* dispara uma chamada AJAX `POST /ai-coach/` e o resultado é renderizado dinamicamente com o anel de pontuação SVG animado.

7.4 AI Spending Analysis

7.4.1 Objetivo

Identificar **desperdícios**, **padrões financeiros ruins** e **riscos** nos gastos do usuário, calculando a economia potencial total em R$/ano.

7.4.2 Modelos de Saída (Pydantic)

```
class WasteItem(BaseModel):
    nome:                str   # Nome do desperd cio
    categoria:           str   # Categoria identificada
    icone:               str   #  cone  Font Awesome
    valor_mensal:        str   # "R$ 89/m  s"
    economia_potencial: str   # "R$ 1.068/ano"
    motivo:              str   # Explica  o do desperd cio
```

```
    acao:                str   # A    o para eliminar

class BadPattern(BaseModel):
    tipo:            str   # Tipo de padr o ruim
    icone:           str
    titulo:          str
    descricao:       str
    impacto_mensal:  str   # R$ calculado
    impacto_anual:   str   # R$ calculado

class Risk(BaseModel):
    nivel:           str   # 'alto', 'medio', 'baixo'
    icone:           str
    titulo:          str
    descricao:       str
    valor_em_risco:str   # R$ em risco
    solucao:         str

class SpendingAnalysis(BaseModel):
    economia_total_mensal: str
    economia_total_anual:  str
    resumo:                str
    desperdicios:          List[WasteItem]   # 2-5 itens
    padroes_ruins:         List[BadPattern] # 2-4 itens
    riscos:                List[Risk]        # 1-3 itens
    quick_wins:            List[str]         # Exatamente 4
```

Listing 7.3: finance/spending_analyzer.py — Modelos Pydantic

7.4.3 Padrões Detectados

- Assinaturas esquecidas ou subutilizadas
- Serviços duplicados (streaming, seguros, etc.)
- Categorias acima dos benchmarks (alimentação $> 15\%$, variáveis $> 20\%$)
- Ausência ou insuficiência de reserva de emergência
- Concentração excessiva em uma categoria de gasto
- Ausência de investimentos com renda disponível positiva

7.5 AI Budget Optimizer

7.5.1 Objetivo

Gerar um **orçamento ideal** para o usuário baseado na Regra 50/30/20 adaptada ao contexto brasileiro, com projeções de patrimônio para 1 e 5 anos.

7.5.2 Regra 50/30/20 Aplicada

Tabela 7.3: Regra 50/30/20 adaptada ao Brasil

Bloco	% Ideal	Categorias	Descrição
Necessidades	$\leq 50\%$	moradia, transporte, saúde, alimentação	Gastos essenciais indispensáveis
Desejos	$\leq 30\%$	lazer, vestuário, eletrônicos	Gastos não essenciais mas legítimos
Futuro	$\geq 20\%$	reservas, investimentos	Construção de patrimônio

7.5.3 Modelos de Saída (Pydantic)

```python
class BudgetCategory(BaseModel):
    categoria:         str   # Nome da categoria
    icone_fa:          str   # Classe FA: 'fa-house', 'fa-car', etc.
    cor:               str   # Cor hex
    valor_atual:       float # Quanto gasta hoje
    valor_ideal:       float # Quanto deveria gastar
    percentual_ideal:  float # % da renda ideal
    diferenca:         float # valor_ideal - valor_atual
    status:            str   # 'acima', 'abaixo', 'ideal'
    justificativa:     str   # Por que este valor é ideal
    acao:              str   # O que fazer para ajustar

class BudgetOptimization(BaseModel):
    renda_mensal:              float
    regra_utilizada:           str          # "50/30/20"
    resumo:                    str
    categorias:                List[BudgetCategory]  # 6-8
    economia_mensal_adicional:float
    projecao_1_ano:            float
    projecao_5_anos:           float        # Com juros compostos 10% a.a.
    insight_principal:         str
    proximos_passos:           List[str]    # Exatamente 3
```

Listing 7.4: finance/budget_optimizer.py — Modelos Pydantic

7.5.4 Cálculo de Projeções

As projeções de patrimônio são calculadas pelo LLM com base em:

- **Projeção 1 ano:** economia_mensal_adicional × 12
- **Projeção 5 anos:** Juros compostos com 10% a.a., aplicados mensalmente sobre a economia acumulada

7.6 AI Investment Assistant

7.6.1 Objetivo

Analisar o portfólio de investimentos do usuário, diagnosticar o **perfil de risco atual vs. ideal**, sugerir realocações e gerar **projeções de patrimônio** para 10 e 20 anos.

7.6.2 Perfis de Risco e Alocações Ideais

Tabela 7.4: Alocação ideal por perfil de risco

Perfil	Renda Fixa	FIIs	Renda Variável	Cripto	Outros
Conservador	60%	20%	15%	0%	5%
Moderado	40%	25%	25%	0%	10%
Arrojado	20%	20%	45%	5%	10%

7.6.3 Modelos de Saída (Pydantic)

```
class RiskProfile(BaseModel):
    perfil_atual:  str   # "Conservador", "Moderado", "
       Arrojado"
    perfil_ideal:  str
    score_atual:   int   # 0-100
    score_ideal:   int
    cor_atual:     str   # Cor hex
    cor_ideal:     str
    diagnostico:   str   # Explica  o do gap entre atual e
       ideal

class AssetAllocation(BaseModel):
    classe:            str   # Classe de ativo
    icone_fa:          str   # 'fa-shield-halved', 'fa-
       building', etc.
    cor:               str
    percentual_atual: float
```

```
    percentual_ideal: float
    valor_atual:      float
    valor_ideal:      float
    diferenca_pct:    float
    justificativa:    str

class InvestmentAnalysis(BaseModel):
    valor_total_investido:    float
    aporte_mensal_atual:      float
    aporte_mensal_ideal:      float    # 20-30% da renda
    percentual_renda_atual:   float
    percentual_renda_ideal:   float
    resumo:                   str
    perfil_risco:             RiskProfile
    alocacao:                 List[AssetAllocation]    # 4-6
    sugestoes:                List[InvestmentSuggestion]  #
       3-5
    projecao_10_anos:         float    # Taxa 12% a.a.
    projecao_20_anos:         float
    projecao_conservadora_10: float    # Taxa 8% a.a.
    insight_premium:          str
    alertas:                  List[str]  # 1-3
```

Listing 7.5: finance/investment_assistant.py — Modelos principais

7.6.4 Fórmula de Projeção

As projeções utilizam a fórmula de valor futuro com juros compostos mensais:

$$FV = PV \cdot (1+r)^n + PMT \cdot \frac{(1+r)^n - 1}{r} \tag{7.1}$$

Onde:

- FV = Valor futuro (patrimônio projetado)
- PV = Patrimônio atual investido
- r = Taxa mensal (12% a.a. $\approx$ 0.949% a.m. | 8% a.a. $\approx$ 0.643% a.m.)
- n = Número de meses (120 para 10 anos, 240 para 20 anos)
- PMT = Aporte mensal ideal

7.7 AI What-if Simulator

7.7.1 Objetivo

Chatbot conversacional que permite ao usuário fazer perguntas hipotéticas (*"e se eu..."*) e recebe respostas com **impactos financeiros calculados** com base nos dados reais,

em formato estruturado com veredicto, impactos e conselho.

7.7.2 Gerenciamento do Histórico de Conversa

```python
def _format_history(self, historico: list) -> str:
    if not historico:
        return "Nenhuma mensagem anterior esta a primeira pergunta."
    lines = []
    for msg in historico[-8:]:  # ltimas 4 trocas (8 mensagens)
        role = "Usu rio" if msg.get("role") == "user" else "Assistente"
        content = msg.get("content", "")
        if role == "Assistente" and isinstance(content, dict):
            content = content.get("resposta", str(content))
        lines.append(f"{role}: {content}")
    return "\n".join(lines)
```

Listing 7.6: finance/whatif_simulator.py — Formatação do histórico

7.7.3 Modelos de Saída (Pydantic)

```python
class ImpactoItem(BaseModel):
    label:     str  # "Redu o na reserva de emerg ncia"
    valor:     str  # "42%", "R$ 1.240/m s", "18 meses"
    tipo:      str  # 'positivo', 'negativo', 'neutro'
    icone_fa: str  # Classe FA: 'fa-arrow-down', 'fa-clock'

class WhatIfResponse(BaseModel):
    veredicto:       str  # 'positivo','cuidado','negativo','critico'
    veredicto_label: str  # ' tima ideia!','Vi vel','Aten o','Invi vel'
    veredicto_cor:   str  # Cor hex correspondente
    resposta:        str  # An lise conversacional em 2-4 frases
    impactos:        List[ImpactoItem]  # 2-4 impactos calculados
    conselho:        str  # Recomenda o espec fica e acion vel
```

Listing 7.7: finance/whatif_simulator.py — Modelos Pydantic

7.7.4 Exemplos de Cenários Suportados

Tabela 7.5: Exemplos de simulações suportadas

Pergunta hipotética	Cálculos realizados
"Se eu comprar um carro?"	Parcela estimada + impacto no saldo + comprometimento da renda
"Se eu trocar de emprego?"	Impacto na renda + efeito cascata nos gastos + viabilidade
"Se eu dobrar meus aportes?"	Projeção de patrimônio adicional em X anos
"E se eu tirar férias caras?"	Impacto na reserva + meses para recompor
"E se eu financiar um imóvel?"	Parcela estimada + comprometimento total da renda
"Se eu tiver um filho?"	Gastos adicionais estimados + impacto no saldo livre

7.7.5 Sistema de Veredictos

Tabela 7.6: Sistema de veredictos do What-if Simulator

Veredicto	Label	Cor	Critério
positivo	Ótima ideia!	`#16A34A`	Melhora a situação financeira
cuidado	Atenção	`#F59E0B`	Viável com planejamento
negativo	Risco Alto	`#F97316`	Impacto negativo significativo
critico	Inviável	`#DC2626`	Comprometeria seriamente as finanças

Capítulo 8

Interface do Usuário

8.1 Sistema de Design

A interface do FinancePro foi construída com um **design system próprio** baseado em CSS Custom Properties (variáveis), sem dependência de frameworks como Bootstrap ou Tailwind.

8.1.1 Paleta de Cores

Tabela 8.1: Paleta de cores do sistema

Variável CSS	Hex	Amostra	Uso
`-primary`	#FF5710		Ações principais, destaques
`-secondary`	#1E1B6E		Sidebar, títulos de seção
`-success`	#16A34A		Positivo, pago, lucro
`-danger`	#DC2626		Erro, negativo, gastos
`-warning`	#D97706		Atenção, reservas
`-purple`	#7C3AED		Investimentos, PRO
`-info`	#0EA5E9		What-if, informações

8.1.2 Componentes Reutilizáveis

Os seguintes componentes CSS são definidos globalmente no `base.html` e usados em todas as páginas:

- **.stat-card** — Card de KPI com accent bar, ícone e valor em destaque
- **.section-card** — Card de seção com header e body padrão
- **.badge** — Etiqueta colorida por categoria (salário, moradia, lazer, etc.)
- **.btn** — Botões com variações: primary, secondary, success, danger, purple, outline, ghost
- **.progress-bar** / **.progress-fill** — Barras de progresso com animação CSS
- **.form-input** / **.form-select** — Campos de formulário com foco em laranja
- **.nav-item** — Item de navegação com estado ativo e hover

- **.nav-badge** — Badge do sidebar (IA, PRO)

8.2 Layout Principal

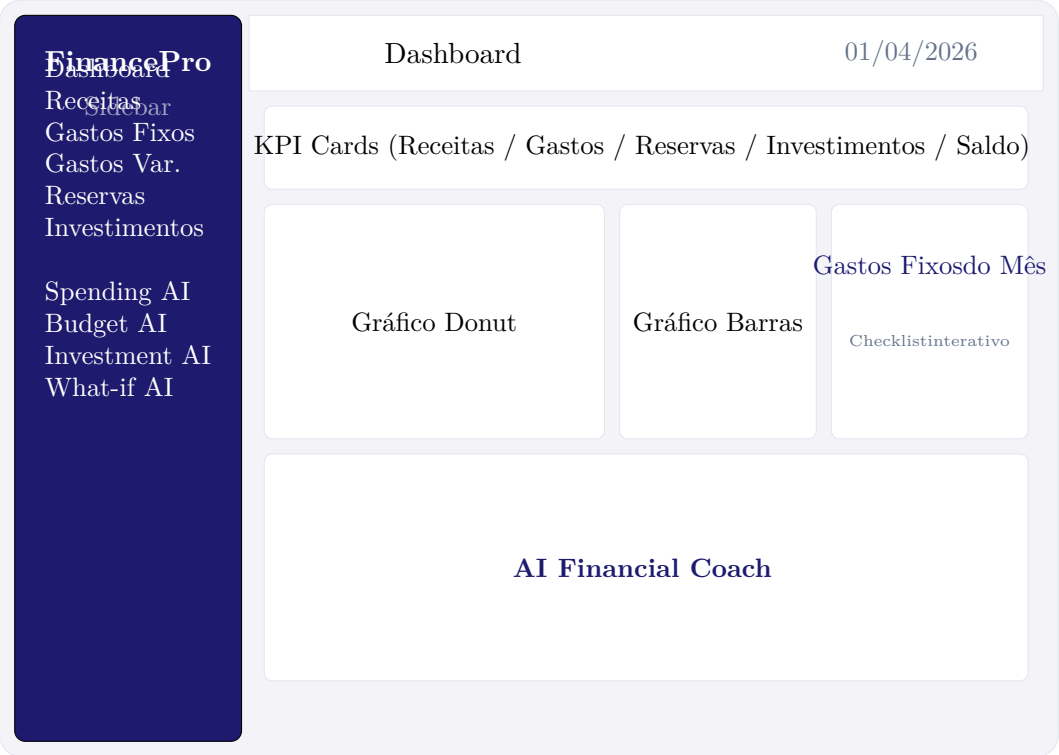


Figura 8.1: Esquema do layout do Dashboard principal

8.3 Template Base

O arquivo `base.html` é a fundação de toda a interface e define:

- **Sidebar fixo** com navegação hierárquica (Visão Geral, Finanças, Inteligência Artificial)
- **Header sticky** com breadcrumb da página atual e data
- **Sistema de mensagens Django** estilizado (success, error, warning, info)
- **Blocos de template:** `title`, `page_title`, `page_sub`, `header_right`, `content`, `extra_js`
- **Responsividade mobile** com overlay e toggle do sidebar

8.4 Páginas de Autenticação

`login.html` Layout split-screen: painel esquerdo com gradiente e informações do produto; painel direito com formulário de login.

`register.html` Formulário de cadastro com validação em tempo real.

`profile.html` Duas seções: edição de dados pessoais e troca de senha, com upload de foto de perfil.

8.5 Interfaces dos Módulos de IA

Dashboard (AI Coach) Painel integrado com anel SVG animado para pontuação, grid de insights e lista numerada de recomendações. Três estados: idle, loading (spinner), results.

Spending Analysis Página dedicada com KPIs de economia potencial, grid de desperdícios com cards coloridos, lista de padrões ruins e grade de quick wins.

Budget Optimizer Exibe categorias com **dupla barra de progresso** (valor atual em cinza, valor ideal em cor da categoria), diferença em R$ e status badge (acima/abaixo/ideal).

Investment Assistant Interface premium com badge PRO, duas barras de perfil de risco animadas, tabela de alocação com gap visual, cards de sugestões coloridos por tipo de ação e três cards de projeção.

What-if Simulator Interface de chatbot com bolhas de mensagem (usuário à direita, IA à esquerda), card estruturado de resposta com verdict bar colorida, grid de impactos e box de conselho. Painel lateral com sugestões clicáveis e legenda de veredictos.

Capítulo 9

Segurança

9.1 Mecanismos Implementados

Tabela 9.1: Recursos de segurança implementados

Mecanismo	Descrição
Autenticação	Sistema de sessões Django com e-mail como identificador único
Autorização	Decorator `@login_required` em todas as views protegidas
CSRF Protection	Middleware Django + token em todas as requisições POST/AJAX
Isolamento de dados	Todas as queries filtram por `user=request.user` (sem acesso cruzado)
Hashing de senhas	`make_password()` via Django (PBKDF2 + SHA256 por padrão)
ORM	Proteção automática contra SQL Injection via Django ORM
Variáveis de ambiente	Chaves sensíveis em `.env` via `python-dotenv`

9.2 Pontos de Atenção para Produção

Aviso de Segurança

Os itens abaixo são aceitáveis em ambiente de desenvolvimento mas **devem ser corrigidos antes do deploy em produção:**

- `DEBUG = True` deve ser alterado para `False`
- `ALLOWED_HOSTS = ['*']` deve listar apenas os domínios reais
- `SECRET_KEY` deve ser gerada com `python -c "from django.core.management.utils import get_random_secret_key; print(get_random_secret_key())"`
- A `OPENAI_API_KEY` deve ser armazenada em cofre de segredos (AWS Secrets Manager, HashiCorp Vault, variáveis de ambiente do servidor)
- SQLite deve ser substituído por PostgreSQL ou MySQL
- Os endpoints de IA (`/*/run/`) não possuem rate limiting — implementar para evitar abuso de API

9.3 Checklist de Segurança para Deploy

`DEBUG = False` no `.env` de produção

`SECRET_KEY` gerado aleatoriamente e único

`ALLOWED_HOSTS` configurado com domínio real

HTTPS/SSL habilitado (Nginx + Certbot)

Banco de dados PostgreSQL configurado

`OPENAI_API_KEY` em variável de ambiente do servidor

Rate limiting nos endpoints `/*/run/` (ex: django-ratelimit)

`SECURE_SSL_REDIRECT = True`

`SESSION_COOKIE_SECURE = True`

`CSRF_COOKIE_SECURE = True`

`.env` adicionado ao `.gitignore`

Capítulo 10

Deploy e Produção

10.1 Requisitos de Servidor

- Sistema operacional: Ubuntu 22.04 LTS (recomendado)
- Python 3.10+
- Nginx (proxy reverso)
- Gunicorn (WSGI server)
- PostgreSQL 14+ (banco de dados de produção)
- Certbot (SSL/TLS gratuito via Let's Encrypt)

10.2 Stack de Produção Recomendada

Listing 10.1: Deploy com Gunicorn + Nginx

```
# 1. Instalar depend ncias do servidor
sudo apt update && sudo apt install python3 python3-pip nginx
    postgresql

# 2. Configurar banco de dados PostgreSQL
sudo -u postgres psql -c "CREATE DATABASE financepro;"
sudo -u postgres psql -c "CREATE USER fpuser WITH PASSWORD '
   senha_segura';"
sudo -u postgres psql -c "GRANT ALL ON DATABASE financepro TO
    fpuser;"

# 3. Instalar depend ncias Python
pip3 install -r requirements_django.txt gunicorn psycopg2-
   binary

# 4. Coletar arquivos est ticos
python manage.py collectstatic --noinput

# 5. Aplicar migra  es
python manage.py migrate

# 6. Iniciar Gunicorn
gunicorn finance_project.wsgi:application \
  --workers 3 \
  --bind 127.0.0.1:8000 \
```

```
  --daemon

# 7. Configurar Nginx como proxy reverso
# /etc/nginx/sites-available/financepro
server {
    listen 80;
    server_name seudominio.com.br;
    location / { proxy_pass http://127.0.0.1:8000; }
    location /static/ { alias /path/to/static/; }
    location /media/ { alias /path/to/media/; }
}
```

10.3 Variáveis de Ambiente de Produção

Listing 10.2: .env de produção (exemplo)

```
SECRET_KEY=gere_uma_chave_aleatoria_de_50_caracteres_aqui
DEBUG=False
ALLOWED_HOSTS=seudominio.com.br,www.seudominio.com.br

# Banco de dados
DATABASE_URL=postgres://fpuser:senha@localhost:5432/
   financepro

# OpenAI
OPENAI_API_KEY=sk-...
```

Capítulo 11

Migrações do Banco de Dados

11.1 Histórico de Migrações

Tabela 11.1: Histórico de migrações do projeto

App	Migração	Alteração
accounts	0001_initial	Criação do modelo User customizado com campos email, names, photo
finance	0001_initial	Criação dos modelos Receita, GastoFixo, GastoVariavel, Reserva, Investimento
finance	0002_gastofixo_pago	Adição do campo `pago` (BooleanField, default=False) ao modelo GastoFixo

11.2 Comandos de Migração

Listing 11.1: Comandos Django para gerenciamento de banco de dados

```
# Verificar status das migra  es
python manage.py showmigrations

# Criar novas migra  es ap s alterar models.py
python manage.py makemigrations

# Aplicar todas as migra  es pendentes
python manage.py migrate

# Aplicar migra  es de um app espec fico
python manage.py migrate finance
python manage.py migrate accounts

# Reverter para uma migra  o espec fica
python manage.py migrate finance 0001_initial
```

Capítulo 12

Guia de Extensão

12.1 Adicionando um Novo Modelo Financeiro

1. Criar o modelo em `finance/models.py` com ForeignKey para User
2. Criar o ModelForm em `finance/forms.py`
3. Adicionar as views CRUD em `finance/views.py` com `@login_required`
4. Registrar as URLs em `finance/urls.py`
5. Criar o template HTML estendendo `base.html`
6. Executar `python manage.py makemigrations && python manage.py migrate`

12.2 Adicionando um Novo Agente de IA

1. Criar o arquivo `finance/nome_agente.py` seguindo o padrão das classes existentes:
   - Definir os modelos Pydantic de saída
   - Criar a classe com `__init__`, `_build_prompt`, `_build_chain`, `analyse`
   - Criar a função `_build_*_context(user)` para montar o contexto
2. Adicionar as views (GET para renderizar, POST para executar) em `views.py`
3. Registrar as URLs em `urls.py`
4. Criar o template HTML com os três estados: idle, loading, results
5. Adicionar o link no sidebar em `base.html`

12.3 Alterando o Modelo de LLM

Para trocar o modelo ou provedor de LLM, basta alterar o `__init__` de qualquer agente:

```
# Usar GPT-4o (mais preciso, mais caro)
self.model = ChatOpenAI(model="gpt-4o", temperature=0.4)

# Usar GPT-3.5-turbo (mais r pido e barato)
self.model = ChatOpenAI(model="gpt-3.5-turbo", temperature
    =0.6)

# Usar Anthropic Claude (via LangChain)
```

```
from langchain_anthropic import ChatAnthropic
self.model = ChatAnthropic(model="claude-3-5-sonnet-20241022"
    )
```

Listing 12.1: Trocar modelo em qualquer agente

Capítulo 13

Referências e Tecnologias

Tabela 13.1: Documentações oficiais

Tecnologia	URL de Referência
Django 4.2	https://docs.djangoproject.com/en/4.2/
LangChain	https://python.langchain.com/docs/
OpenAI API	https://platform.openai.com/docs/
Pydantic v2	https://docs.pydantic.dev/latest/
Chart.js 4.4	https://www.chartjs.org/docs/latest/
Font Awesome 6.5	https://fontawesome.com/icons
python-dotenv	https://pypi.org/project/python-dotenv/
Gunicorn	https://gunicorn.org/

FinancePro

Documentação Técnica — v1.0

Desenvolvido com Django · LangChain · OpenAI GPT-4o-mini

Gestão financeira inteligente para o mercado brasileiro

Tecnologias Principais

Django 4.2	**LangChain**	**OpenAI**	**Pydantic v2**	**Chart.js**
Python 3.x	GPT-4o-mini	SQLite3	Font Awesome	Inter